

基本情報技術者試験 科目B対策

擬似言語とAIで学ぶ アルゴリズム入門

はじめての変数から、配列のトレースまで

「答えを聞く」のではなく、
「分からない場所を見つける」ためにAIを使おう。

初稿・内容確認版

はじめに

この教材は、基本情報技術者試験の科目Bで出題されるアルゴリズムとプログラミングを、プログラミング未経験者が擬似言語から学ぶための入門書です。

科目Bの問題を解くために、難しいプログラム言語を最初から覚える必要はありません。まず必要なのは、次の三つの力です。

1. 擬似言語を上から順番に読む力
2. 変数や配列の値がどう変化するかを追う力
3. 分からない箇所を言葉にして、質問する力

本書では、フローチャートの記号や作図方法は扱いません。処理の基本となる順次・選択・繰返しを、最初から擬似言語で学びます。

また、本書には生成AIへ質問するための例文を掲載しています。AIに正解を直接聞くのではなく、考え方の整理、ヒント、トレースの確認、間違いの発見に利用します。

本書の到達目標

- 変数・代入・配列を説明できる
- 選択処理と繰返し処理を読める
- 短い擬似言語をトレースできる
- 合計・件数・最大値を求める処理を理解できる
- AIを使って、自分で学習を続けられる

この教材の使い方

各節は、次の順番で進みます。

1 概念を読む 何をする仕組みか理解する	2 例を追う 値の変化を手で記録する	3 自力で解く 最初からAIに聞かない	4 AIに質問する 分からない箇所だけ尋ねる	5 解き直す AIを閉じてもう一度解く
-----------------------------------	---------------------------------	----------------------------------	-------------------------------------	----------------------------------

大切なルール

AIの説明は、いつも正しいとは限りません。本書の解説や公式資料と照合し、自分で値を追って確認してください。「AIが言ったから正しい」ではなく、「自分でも同じ結果を確認できたから正しい」と判断します。

目次

1. AIを学習相手にする
2. アルゴリズムと擬似言語
3. 変数・データ型・代入
4. 順次処理
5. 選択処理
6. 繰り返し処理
7. 配列
8. トレースの技術
9. 総合演習
10. 解答・解説
11. 研修後の学習ロードマップ

第0章 AIを学習相手にする

0.1 AIは「正解を出す機械」ではない

生成AIへ問題をそのまま入力し、「答えを教えて」と頼めば、短時間で答えらしきものが表示されます。しかし、それだけでは自分で問題を解く力は身に付きません。

本書では、AIを次の四つの役割で使います。

AIの役割	何をしてもらうか	使用する場面
整理役	入力・処理・出力を分ける	問題文の意味が分からない
ヒント役	次に見るべき場所を示す	解き始められない
点検役	最初に間違えた箇所を示す	自分の考えに自信がない
練習相手	難易度を調整した類題を作る	もう一問練習したい

避けたい質問

問題文を渡さずに「答えを教えて」とだけ頼むことや、考える前に「完成した擬似言語を作って」と頼むこと

正解だけを受け取ると、どこで考えられなくなったのかが分かりません。

学習につながる質問

「答えはまだ示さず、最初に確認すべき変数を一つ教えてください」

「私のトレース表で、最初に間違っている行だけを指摘してください」

0.2 AIに質問する前の30秒

質問を入力する前に、次の三点を書き出します。

AIへ聞く前の整理メモ	
1 分かっていること	ここに書く -----
2 分からないこと	ここに書く -----
3 自分で試したこと	ここに書く -----

例えば、次のように書きます。

記入例	
1 分かっていること	totalが合計を保存する変数であることは分かります。
2 分からないこと	<code>total ← total + A[i]</code> の右辺で、どの値を使うのか分かりません。
3 自分で試したこと	iが1の場合までは表に書きました。

ここまで書くと、AIへの質問が具体的になります。また、自分がどこまで理解できているかも見えるようになります。

0.3 問題をAIへ渡す

AIに質問するには、まずAIが読める形で問題を渡す必要があります。方法は次の三つです。

1 文章をコピーして貼る WebページやPDFで文字を選択できる場合	2 スクリーンショットを添付する 図・表・擬似言語を含む問題の場合	3 必要な部分を手で入力する コピーも画像添付もできない場合
--	---	--------------------------------------

方法1 問題文をコピーして貼り付ける

問題文、擬似言語、選択肢をまとめてコピーし、質問の前に貼り付けます。長い問題では、次の見出しを付けるとAIが内容を区別しやすくなります。

AIへ貼り付ける形	
問題文	ここに問題文を貼る
擬似言語	ここに擬似言語を貼る
選択肢	ここに選択肢を貼る
自分が分からないところ	例: 3行目で、totalにどの値が入るのか分からない

方法2 スクリーンショットを添付する

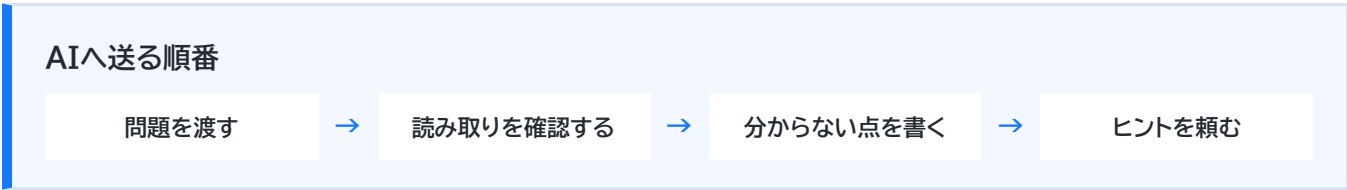
文字を選択できない場合は、問題全体が読めるスクリーンショットを添付します。問題文、擬似言語、選択肢が途中で切れないようにしてください。

画像を添付したら、最初に読み取り確認をする

AIは、画像内の添字、記号、数字を読み間違えることがあります。すぐに解かせず、「画像から読み取った問題文と擬似言語をそのまま書き出してください」と頼み、元画像と見比べます。

方法3 必要な部分を手で入力する

画像を添付できない場合は、問題の条件と擬似言語を入力します。すべてを打ち直すのが大変なら、分からない行と、その行で使う変数の初期値だけでも構いません。



0.4 本書で使う基本プロンプト

次の例文は、問題文や画像をAIへ渡し、内容が正しく読み取られたことを確認したあとに続けて送ります。

AI

AIへの質問例 | ヒントが欲しいとき

あなたはプログラミング未経験者の学習支援者です。先ほど渡した問題について、正解や完成した擬似言語はまだ示さないでください。最初に確認すべきことを一つだけ、私に質問してください。私が答えたら次の質問へ進んでください。

AI

AIへの質問例 | 自分の考えを点検するとき

これから私の考え方を説明します。正解を先に示さず、論理が最初に成立しなくなる箇所だけを指摘してください。その理由は一文で説明してください。

AI

AIへの質問例 | 類題を作るとき

先ほど渡した問題と同じ考え方で解ける、数字だけを変更した練習問題を一問作ってください。問題だけを提示し、解答と解説は私が回答するまで表示しないでください。

0.5 AIの回答を確認する

AIから回答を得たら、次の順番で確認します。

1. 問題文に書かれていない条件を勝手に追加していないか
2. 配列の先頭の添字を取り違えていないか
3. 繰返しの開始値・終了値を取り違えていないか
4. 代入前の値と代入後の値を混同していないか
5. 自分でトレースして同じ結果になるか

個人情報・会社情報を入力しない

氏名、メールアドレス、顧客情報、公開されていない会社資料、パスワードなどはAIへ入力しません。この教材の練習問題のように、公開しても問題のない内容だけを使います。

第1章 アルゴリズムと擬似言語

1.1 アルゴリズムとは

アルゴリズムとは、目的を達成するための**処理手順**です。

身近なたとえ

アルゴリズムは「料理のレシピ」と同じ

カレーを作るとき、材料だけを渡されても完成しません。「切る → 炒める → 煮る」のように、作業の順番が必要です。コンピュータにも、同じように具体的な手順を一つずつ伝えます。



例えば、「三つの数の中から最大の数を見つける」という目的に対して、次の手順を考えられます。

1. 一つ目の数を、現在の最大値として覚える
2. 二つ目の数が現在の最大値より大きければ、最大値を更新する
3. 三つ目の数も同じように比較する
4. 最後に残った最大値を表示する

人間が何となく行っている判断を、コンピュータが実行できる順序に分解したものがアルゴリズムです。

例: 3、10、7の中から最大値を探す

3

最初の暫定1位

10

3より大きいので、暫定1位を10へ交代

7

10より小さいので、そのまま

最後の暫定1位「10」が答え

1.2 良いアルゴリズム

アルゴリズムには、少なくとも次の性質が必要です。

- 手順が明確である
- 同じ入力に対して、決められた結果になる
- いつか処理が終了する
- 実行できない曖昧な指示がない

「いい感じに並べる」「適切な回数だけ繰り返す」という表現は、人間には伝わってもコンピュータには伝わりません。

人には通じるかもしれない

「数字をいい感じに並べて」

何を基準に？ 大きい順？ 小さい順？

コンピュータにも伝わる

「左から右へ、小さい順に並べる」

基準と終了地点が明確

1.3 擬似言語とは

擬似言語は、アルゴリズムをプログラムに近い形で表現するための記述方法です。特定のプログラミング言語に依存せず、処理の考え方を表します。



本書では、次のような表記を使います。

```
整数型: price
整数型: count
整数型: total

price ← 150
count ← 3
total ← price × count
表示する(total)
```

上から一行ずつ実行すると、`total` には450が入り、450が表示されます。

本書の表記について

試験問題では、問題文の中で配列の添字や関数の仕様が定義されることがあります。必ず問題文の定義を優先してください。本書の表記は、初学者が読みやすいように整えた学習用表記です。

1.4 処理を作る三つの基本構造

複雑なアルゴリズムも、基本的には次の三つを組み合わせて作ります。

1

順次

上から順番に行う

例: 服を着てから靴を履く

2

選択

条件で行動を変える

例: 雨なら傘を持つ

3

繰り返し

同じことを何度か行う

例: 全員分の出席を取る

順次

上から下へ、決められた順番で処理します。

```
a ← 5
b ← 10
c ← a + b
```

選択

条件によって、実行する処理を分けます。

```
if (score >= 60)
    "合格"を表示する
else
    "不合格"を表示する
endif
```

繰り返し

条件や回数に従って、同じ処理を繰り返します。

```
for (iを1から5まで1ずつ増やす)
    iを表示する
endfor
```

理解チェック

「合計が100以上なら割引し、商品ごとに同じ計算を繰り返す」という処理には、三つの基本構造のうち何が含まれますか。

第2章 変数・データ型・代入

2.1 変数は値を覚える場所

プログラムでは、計算途中の値を覚えておく必要があります。その保管場所が**変数**です。

変数には名前を付けます。例えば、合計を保存する変数なら `total`、件数なら `count` のように、役割が分かる名前を使います。

変数は「名前付きの箱」

total

0

箱の名前: ``total``

箱の中身: ``0``

箱に入れられるもの: 整数

```
整数型: total
total ← 0
```

この例では、整数を保存する変数 `total` を用意し、最初の値として0を入れています。

2.2 データ型

データ型は、変数にどのような値を保存するかを表します。

データ型は、箱に貼る「中に何をに入れてよいか」というラベルです。整数用の箱に文章を入れないように、保存する値の種類をあらかじめ決めます。

データ型	保存する値の例	使用例
整数型	<code>10</code> 、 <code>-3</code> 、 <code>0</code>	件数、添字、合計
実数型	<code>3.14</code> 、 <code>18.5</code>	平均、身長、割合
文字型	<code>'A'</code> 、 <code>'?'</code>	一文字の記号
文字列型	<code>"合格"</code> 、 <code>"Tokyo"</code>	名前、メッセージ
論理型	<code>true</code> 、 <code>false</code>	条件が成立するか

数字と文字列は別物

整数の`10`は計算に使えますが、文字列の`"10"`は文字として扱われます。問題文で指定されたデータ型を確認しましょう。

2.3 代入は右から左へ読む

代入とは、計算結果や値を変数へ入れる操作です。

```
total ← total + 5
```

右側を先に計算

$3 + 5 = 8$

→

左側の箱へ入れる

total = 8

これは、次の順番で実行します。

1. 現在の **total** の値を取り出す
2. その値に5を足す
3. 計算結果を **total** へ入れ直す

数学の等式ではありません。「左辺と右辺が等しい」とは読みません。

例

実行前の **total** が3の場合を考えます。

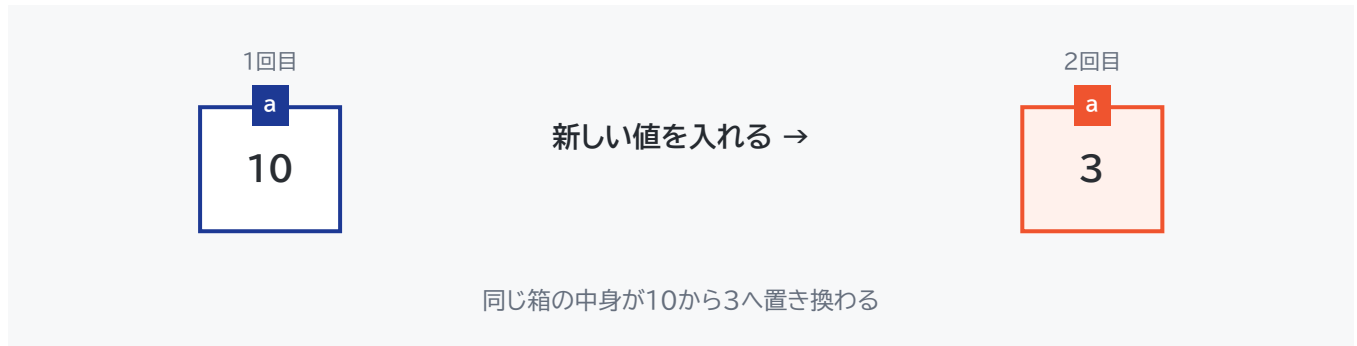
実行する文	右辺の計算	実行後の total
<code>total ← total + 5</code>	<code>3 + 5</code>	8

2.4 値は上書きされる

変数へ新しい値を代入すると、以前の値は失われます。

```
a ← 10  
a ← 3
```

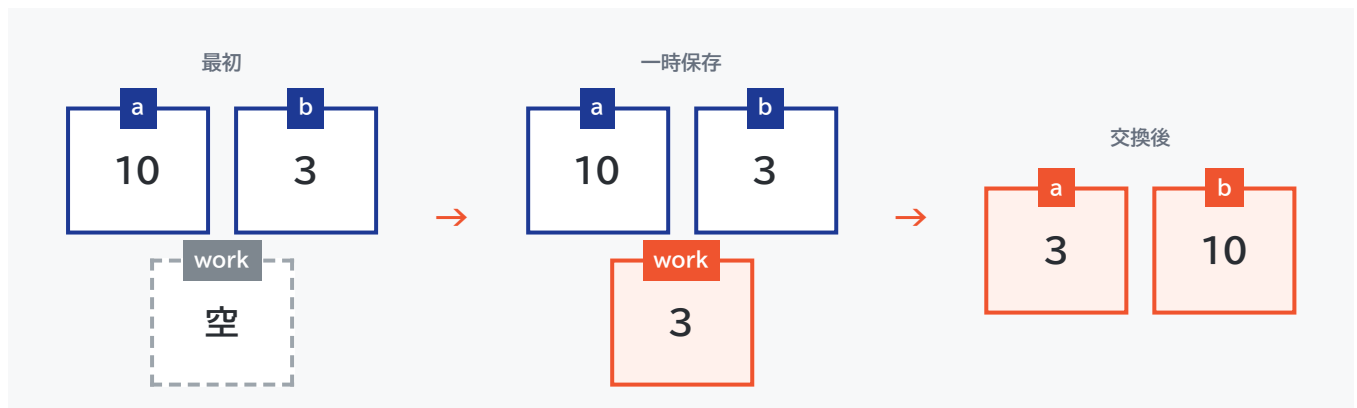
実行後の **a** は3です。最初に入れた10は残りません。



2.5 二つの変数の値を交換する

a に10、**b** に3が入っているとします。二つの値を交換するには、一時的に値を保存する変数が必要です。

二つのコップの飲み物を交換するとき、空のコップが一つ必要なのと同じです。**work** は、値を一時的に避難させる空のコップです。



整数型: work

```
work ← b  
b ← a  
a ← work
```

実行後	a	b	work
初期状態	10	3	未定義
work ← b	10	3	3
b ← a	10	10	3
a ← work	3	10	3

よくある間違い

```
a ← b
b ← a
```

一行目を実行した時点で、`a`に入っていた10が失われます。二行目では、どちらも3になってしまいます。

演習2-A 代入を追う

難易度 ★☆☆

次の処理が終わったとき、total の値はいくつですか。

整数型: total

```
total ← 2
total ← total + 3
total ← total × 2
```

実行した文	実行後の total
total ← 2	
total ← total + 3	
total ← total × 2	

AI

AIへの質問例 | 値の変化を追えないとき

最終的な答えはまだ教えないでください。各行の右辺を先に計算するために、私へ一問ずつ質問してください。私の回答が間違っている場合は、その行だけを指摘してください。

演習2-B 値を交換する

難易度 ★☆☆

`x` に7、`y` に12が入っています。実行後に `x` が12、`y` が7となるように、空欄を埋めてください。

整数型: `work`

```
work ← [ ① ]
```

```
x ← [ ② ]
```

```
y ← [ ③ ]
```

説明してみよう

なぜ一時変数`work`が必要なのかを、プログラミングを知らない人へ説明してください。

第3章 順次処理

3.1 上から順番に実行する

順次処理は、文を上から下へ一度ずつ実行する、最も基本的な構造です。

① リンゴの単価を150円として覚える



② みかんの単価を80円として覚える



③ それぞれの代金を計算して足す



④ 合計金額を表示する

整数型: apple

整数型: orange

整数型: total

apple ← 150

orange ← 80

total ← apple × 3 + orange × 4

表示する(total)

この処理は、150円のリンゴを3個、80円のみかんを4個買ったときの合計金額を求めています。

3.2 入力・処理・出力で整理する

問題文を読んだら、最初に次の三つへ分けます。



分類	意味	例
入力	最初に与えられる値	単価、個数
処理	入力を使って行う計算	単価×個数、合計
出力	最後に求めるもの	合計金額

AI

AIへの質問例 | 問題文を整理するとき

完成した擬似言語は作らないでください。次の問題文から、入力・処理・出力を分けるための質問を、一問ずつ私にしてください。私が答えた後、不足があれば指摘してください。

演習3-A 代金を計算する

難易度 ★☆☆

一つ150円のリンゴを3個、一つ80円のみかんを4個買います。合計金額を計算して表示するように、空欄を埋めてください。

整数型: apple
整数型: orange
整数型: total

apple ← [①]
orange ← [②]
total ← [③]
表示する(total)

自分の説明

自分の言葉で書く	
変数 <code>apple</code> の役割	-----
変数 <code>orange</code> の役割	-----
変数 <code>total</code> の役割	-----

ペアまたはAIへ説明しよう

自分が作った式を、どの部分が「リンゴの代金」「みかんの代金」「全体の合計」なのかに分けて説明してください。聞き手は、答えを教えるのではなく、変数の役割について質問します。

演習3-B 消費税込みの金額

難易度 ★★★

商品の税抜価格 `price` と税率 `rate` が与えられています。税込価格を求めて表示する処理を作成してください。小数点以下の扱いは考えず、実数型で計算します。

```
実数型: price
実数型: rate
実数型: taxIncluded

price ← 1200
rate ← 0.1

// ここに処理を書く
```

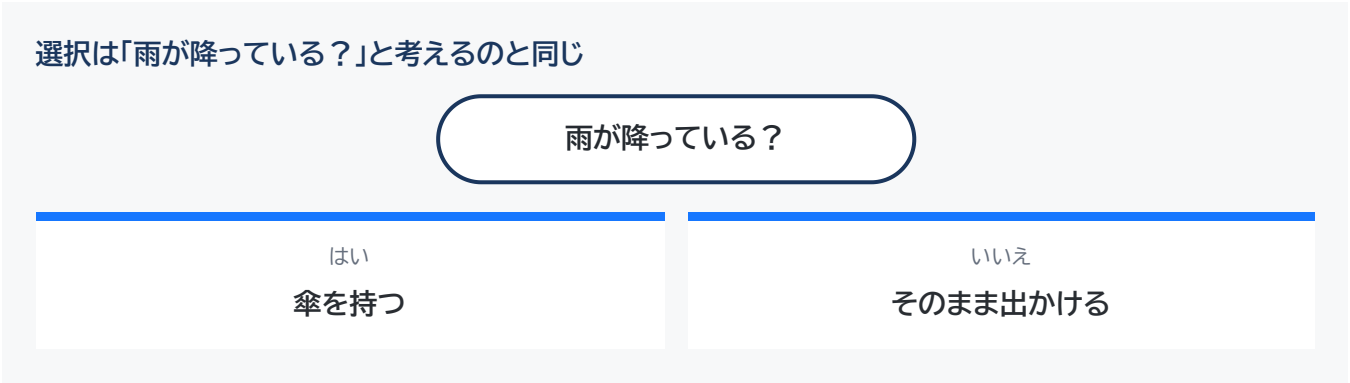
AI AIへの質問例 | 式を点検するとき

私が作った税込価格の計算式を確認してください。正しい式を先に示さず、「税額を求めている式」か「税込価格を求めている式」という観点だけで指摘してください。

第4章 選択処理

4.1 条件によって処理を分ける

選択処理は、条件が成立するかどうかによって、実行する処理を変えます。



```
if (score >= 60)
    "合格"を表示する
else
    "不合格"を表示する
endif
```

条件 `score >= 60` が成立すれば「合格」、成立しなければ「不合格」を表示します。

4.2 比較演算子

演算子	意味	例
<code>=</code>	等しい	<code>x = 10</code>
<code>≠</code>	等しくない	<code>x ≠ 10</code>
<code>></code>	より大きい	<code>x > 10</code>
<code><</code>	より小さい	<code>x < 10</code>
<code>>=</code>	以上	<code>x >= 10</code>
<code><=</code>	以下	<code>x <= 10</code>

「18より大きい」と「18以上」は異なります。境界となる値を確認しましょう。

「60点以上」の境界を見る

59

60

61

条件を満たさない

ここから条件を満たす

4.3 複数の条件

二つ以上の条件を組み合わせるときは、論理演算を使います。

and

学生証も必要
予約票も必要

両方そろって通れる

or

学生証または
運転免許証

どちらか一方で通れる

not

「雨ではない」
条件を反対にする

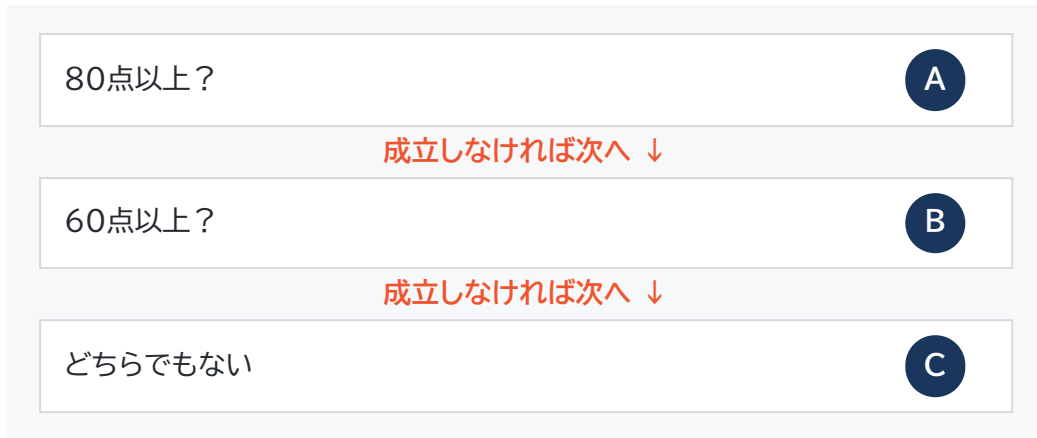
論理演算	成立する条件
and	両方の条件が成立
or	少なくとも一方が成立
not	条件が成立しない

```
if (age >= 18 and age < 65)
    "対象"を表示する
endif
```

4.4 三つ以上に分ける

```
if (score >= 80)
    "A"を表示する
elseif (score >= 60)
    "B"を表示する
else
    "C"を表示する
endif
```

上から条件を確認し、最初に成立した処理だけを実行します。そのため、条件を書く順番が重要です。



順番を逆にするとどうなるか

```
if (score >= 60)
    "B"を表示する
elseif (score >= 80)
    "A"を表示する
endif
```

`score`が90でも、最初の`score >= 60`が成立するため「B」と表示されます。

演習4-A 偶数・奇数を判定する

難易度 ★☆☆

整数 x が偶数なら「偶数」、そうでなければ「奇数」と表示します。空欄を埋めてください。 $x \bmod 2$ は、 x を2で割った余りです。

```
if ([ ① ])
    "偶数"を表示する
else
    "奇数"を表示する
endif
```

演習4-B BMIを判定する

難易度 ★★★

身長 `height` (m)と体重 `weight` (kg)からBMIを求め、次の基準で結果を表示します。

BMIの計算式

BMI = 体重 ÷ (身長 × 身長)

18.5未満 低体重	18.5以上25未満 標準体重	25以上 肥満
---------------	--------------------	------------

```
実数型: height
実数型: weight
実数型: bmi
文字列型: result

height ← 入力する()
weight ← 入力する()
bmi ← weight ÷ (height × height)

if ([ ① ])
    result ← "低体重"
elseif ([ ② ])
    result ← "標準体重"
else
    result ← "肥満"
endif

表示する(result)
```

境界値を確認する

bmi	期待する結果
18.49	
18.50	
24.99	
25.00	

AIへの質問例 | 条件漏れを点検するとき

完成した条件式は表示しないでください。私が書いたBMI判定について、18.5と25.0の境界値を使って判定漏れや重複がないか確認し、問題がある条件だけを指摘してください。

説明してみよう

二つ目の条件を`bmi < 25`だけで書ける理由を説明してください。最初の`if`が成立しなかった時点で、どの条件が分かっているでしょうか。

第5章 繰り返し処理

5.1 同じ処理を繰り返す

繰り返し処理は、決められた回数または条件が成立している間、同じ処理を実行します。

5人の出席を確認する



「名前を呼ぶ → 返事を記録する」を5回繰り返す

```
for (iを1から5まで1ずつ増やす)
  iを表示する
endfor
```

この処理は、1、2、3、4、5を順番に表示します。

5.2 回数が決まっている繰り返し

`for` は、繰り返す回数が分かっている場合に向いています。



整数型: `i`

```
for (iを1から10まで1ずつ増やす)
  iを表示する
endfor
```

5.3 条件で続ける繰返し

`while` は、条件が成立している間、処理を繰り返します。

身近なたとえ: 行列に人がいる間は受付を続ける

`for`: 最初から「10人」と分かっているとき

`while`: あと何人来るか分からず、「待っている人がいる間」続けるとき

整数型: `i`

```
i ← 1
while (i <= 5)
  iを表示する
  i ← i + 1
endwhile
```

`while` を読む順番

1. 繰返しの前に初期値を確認する
2. 条件式を確認する
3. 条件が成立したら処理を実行する
4. 変数の更新を確認する
5. 条件式へ戻る

無限ループに注意

`i ← i + 1` がなければ、`i` は1のままです。条件 `i <= 5` が常に成立し、処理が終わりません。繰返しでは「いつ条件が成立しなくなるか」を確認します。

5.4 合計を求める

1から5までの合計を求めます。

「total」は、数字を貯める貯金箱

$$\boxed{0} + 1 \quad \boxed{1} + 2 \quad \boxed{3} + 3 \quad \boxed{6} + 4 \quad \boxed{10} + 5 \quad \boxed{15}$$

整数型: i

整数型: total

```
total ← 0
```

```
for (iを1から5まで1ずつ増やす)
```

```
    total ← total + i
```

```
endfor
```

表示する(total)

i	実行前の total	total + i	実行後の total
1	0	1	1
2	1	3	3
3	3	6	6
4	6	10	10
5	10	15	15

累積する変数

「total」のように、繰返しのたびに値を加えていく変数を累積用の変数として使います。合計の初期値は通常0です。

5.5 件数を数える

1から10までの整数のうち、偶数の件数を数えます。

「count」は、条件に合ったときだけ押すカウンター



色の付いた偶数だけで、カウンターを1増やす

整数型: i

整数型: count

```
count ← 0
```

```
for (iを1から10まで1ずつ増やす)
```

```
  if (i mod 2 = 0)
```

```
    count ← count + 1
```

```
  endif
```

```
endfor
```

表示する(count)

count は、条件が成立した場合だけ1増えます。

演習5-A 合計と平均

難易度 ★★★

1から10までの整数を順番に表示し、最後に合計と平均を表示します。空欄を埋めてください。

```
整数型: i
整数型: total
実数型: average
```

```
total ← [ ① ]

for (iを[ ② ]から[ ③ ]まで1ずつ増やす)
  iを表示する
  total ← [ ④ ]
endfor

average ← [ ⑤ ]
表示する(total)
表示する(average)
```

AI

AIへの質問例 | 繰返しを追うとき

空欄の答えは教えないでください。`i`、実行前の`total`、実行後の`total`を記録する空のトレース表を作ってください。最初の二回は私が埋めるので、間違っているセルだけを指摘してください。

演習5-B 処理回数を数える

難易度 ★★★

次の処理で、処理①と処理②はそれぞれ何回実行されますか。

```
整数型: i

i ← 1

while (i <= 5)
  // 処理①
  if (i mod 2 = 1)
    // 処理②
  endif
  i ← i + 1
endwhile
```

i	i <= 5	処理①	i mod 2 = 1	処理②
1				
2				
3				
4				
5				
6				

AI

AIへの質問例 | 自分の表を点検するとき

実行回数の正解はまだ教えないでください。私が作成した表を確認し、条件判定と処理実行を混同している最初の行だけを指摘してください。

第6章 配列

6.1 配列は同じ型の値をまとめる

配列は、同じデータ型の複数の値を、一つの名前で管理する仕組みです。

配列は「番号付きのロッカー」

A[1]	A[2]	A[3]
5	10	3

配列名は建物の名前、添字はロッカー番号、値は中に入っている荷物です。

整数型の配列: $A \leftarrow \{5, 10, 3\}$

本書のこの例では、添字を1から始めます。

要素	A[1]	A[2]	A[3]
値	5	10	3

A は配列全体の名前、A[2] は二番目の要素を表します。

添字の開始位置を確認する

問題によって、配列の添字が0から始まる場合と1から始まる場合があります。本書の例を暗記するのではなく、問題文の定義を必ず確認してください。

6.2 配列の要素を順番に表示する

```
整数型: i
整数型の配列: A ← {3, 8, 2, 5}

for (iを1から4まで1ずつ増やす)
  A[i]を表示する
endfor
```

`i` が1、2、3、4と変化するため、`A[1]`、`A[2]`、`A[3]`、`A[4]` を順番に参照します。



6.3 配列の合計

```
整数型: i
整数型: total
整数型の配列: A ← {3, 8, 2, 5}

total ← 0

for (iを1から4まで1ずつ増やす)
  total ← total + A[i]
endfor

表示する(total)
```


i	A[i]	実行前の total	実行後の total
1	3	0	3
2	8	3	11
3	2	11	13
4	5	13	18

$$0 + A[1] \text{ の } 3 = 3 + A[2] \text{ の } 8 = 11 + A[3] \text{ の } 2 = 13 + A[4] \text{ の } 5 = 18$$

6.4 配列の最大値

整数型: i
 整数型: max
 整数型の配列: A ← {2, 5, 1, 9, 8, 10, 7, 3, 6, 4}

max ← A[1]

```
for (iを2から10まで1ずつ増やす)
  if (A[i] > max)
    max ← A[i]
  endif
endfor
```

表示する(max)

最初の要素 A[1] を、現在の最大値として max へ入れます。その後、残りの要素を一つずつ比較します。

「max」は「暫定チャンピオン」



今のチャンピオンより大きい値が現れたときだけ、「max」を交代する

なぜ最大値の初期値を0にしないのか

配列の値がすべて負の数の場合、0は配列に存在しないのに最大値として残ってしまいます。最初の要素を初期値にすれば、配列内の値から最大値を選べます。

演習6-A 10個の合計

難易度 ★★☆☆

配列 `A` には、添字1から10までの10個の整数が入っています。合計を求めるように空欄を埋めてください。

```
整数型: i
整数型: total
整数型の配列: A

total ← [ ① ]

for (iを[ ② ]から[ ③ ]まで1ずつ増やす)
    total ← [ ④ ]
endfor

表示する(total)
```

AI

AIへの質問例 | 添字と値を整理するとき

空欄の答えは表示しないでください。`i`、`A[i]`、`total`がそれぞれ何を表しているかを確認する質問を、一問ずつ出してください。

演習6-B 最大値の変化を追う

難易度 ★★☆☆

次の配列を使って、最大値を求める処理をトレースしてください。

```
A ← {2, 5, 1, 9, 8, 10, 7, 3, 6, 4}
```

i	A[i]	比較前の max	A[i] > max	比較後の max
初期化	2	—	—	2
2	5			
3	1			
4	9			
5	8			
6	10			
7	7			
8	3			
9	6			
10	4			



AIへの質問例 | 自分のトレースを確認するとき

最終的な最大値は教えないでください。私の表で、`max`を更新する必要があるのに更新していない最初の行、または更新する必要があるのに更新した最初の行だけを指摘してください。

第7章 トレースの技術

7.1 コードを眺めるだけでは解けない

科目Bでは、擬似言語を見て「何となく分かった」と感じて、選択肢を正しく選べないことがあります。値の変化を紙に書き、処理を実行した結果を確認する必要があります。

この作業をトレースといいます。

1回目 `total ← 0 + 1`

2回目 `total ← 1 + 2`

3回目 `total ← 3 + 3`

頭の中だけで追わない

→ 「何回目か」「計算前」「計算後」を表へ書く。トレースは、プログラムの実況中継です。

7.2 最初に表の列を決める

すべての変数を書く必要はありません。次を目安に列を作ります。

いつ？

`i`

何回目か

何を使う？

`A[i]`

現在の要素

どう変わる？

`total`

実行前と実行後

- 繰り返しを制御する変数
- 条件式で参照する変数
- 処理によって値が更新される変数
- 配列の場合は、添字と現在参照する要素

例

```
整数型: i
整数型: total

total ← 0

for (iを1から3まで1ずつ増やす)
    total ← total + i × 2
endfor
```

i	i × 2	実行前の total	実行後の total
1	2	0	2
2	4	2	6
3	6	6	12

7.3 「いつの値か」を明確にする

次の二つは意味が異なります。

- 文を実行する前の値
- 文を実行した後の値

トレース表の見出しに「実行前」「実行後」と書くと、混乱を防げます。

7.4 条件判定も一行として記録する

選択処理や繰返しでは、処理本体だけでなく条件判定も行われます。

```
i ← 1

while (i <= 3)
    iを表示する
    i ← i + 1
endwhile
```

i が4になったときも、条件 $i \leq 3$ は一度判定されます。ただし、処理本体は実行されません。

i	i <= 3	表示	更新後の i
1	true	1	2
2	true	2	3
3	true	3	4
4	false	—	—

7.5 AIにトレースさせる前に

AIへ「トレース表を作って」と頼むだけでは、AIが作った表を眺めて終わってしまいます。次の順序で使います。

1. 自分で必要な列を決める
2. 最初の一回分を自分で記入する
3. AIには列の過不足、または最初の誤りだけを確認させる
4. 残りを自分で埋める
5. 最後に解説と照合する

AI

AIへの質問例 | 表の設計を確認するとき

この擬似言語をトレースするために、私は表の列を「i、A[i]、total」としました。答えや完成表は表示せず、追跡に不足している列、または不要な列があれば、その理由だけを教えてください。

演習7-A 最初の誤りを見つける

難易度 ★★★

次の処理を考えます。

```
整数型: i
整数型: total

total ← 1

for (iを1から4まで1ずつ増やす)
    total ← total × i
endfor
```

ある受講者が次の表を作りました。最初に間違っている行を見つけ、正しい値を記入してください。

i	実行前の total	total × i	実行後の total
1	1	1	1
2	1	2	2
3	2	5	5
4	5	20	20

AI

AIへの質問例 | 自分の説明をレビューするとき

私は「iが3の行で、 2×3 を5と計算していることが最初の誤り」と説明しました。正解を追加で示さず、この説明が論理的に成立しているかだけを評価してください。

第8章 総合演習

この章では、複数の知識を組み合わせます。最初からAIへ入力せず、問題ごとに次の欄を埋めてください。

問題を解く前の確認欄	
使われている変数	-----
使われている配列	-----
選択処理	-----
繰返し処理	-----
求めている結果	-----

演習8-A 条件に合う値の件数

難易度 ★★☆☆

配列 **A** には10個の整数が入っています。5以上の値がいくつあるかを数えて表示します。空欄を埋めてください。

```
整数型: i
整数型: count
整数型の配列: A

count ← [ ① ]

for (iを1から10まで1ずつ増やす)
  if ([ ② ])
    [ ③ ]
  endif
endfor

表示する(count)
```


空欄①～③を私が埋めました。完成コードを示さず、各空欄が「初期化」「条件判定」「件数の更新」のどの役割になっているかを確認してください。誤りがあれば、最初の空欄だけを指摘してください。

演習8-B 最大値とその位置

難易度 ★★★

配列 **A** の最大値と、その値が入っている添字を表示します。最大値が複数ある場合は、最初に現れる位置を表示します。

```
整数型: i
整数型: max
整数型: maxIndex
整数型の配列: A

max ← A[1]
maxIndex ← 1

for (iを2からAの要素数まで1ずつ増やす)
  if ([ ① ])
    max ← [ ② ]
    maxIndex ← [ ③ ]
  endif
endfor

表示する(max)
表示する(maxIndex)
```

考えるポイント

- 条件を `>=` にすると、最大値が複数ある場合に何が起きるか
- `max` を更新するタイミングと `maxIndex` を更新するタイミング
- 繰返しを2から始める理由

AI

AIへの質問例 | 条件の違いを考える

`A[i] > max`と`A[i] >= max`の違いについて、具体的な配列を一つ使って質問形式で説明してください。最終的な空欄の答えは表示しないでください。

演習8-C 記号を正方形に表示する

難易度 ★★★

整数 `num` が入力されます。`num` 行、各行に `num` 個の `*` を表示し、正方形を作ります。改行せずに文字を表示する処理を 横に表示する()、改行する処理を 改行する() とします。

```
整数型: num
整数型: i
整数型: j

num ← 入力する()

for ([ ① ])
  for ([ ② ])
    [ ③ ]
  endfor
  [ ④ ]
endfor
```

`num` が3の場合の出力:

```
***
***
***
```

AI

AIへの質問例 | 入れ子の繰返し

完成した擬似言語は示さないでください。外側の繰返しと内側の繰返しが、それぞれ「行数」と「一行の文字数」のどちらを担当するか、私が説明できるように質問してください。

演習後の振り返り

振り返る項目	記入欄
最初に分からなかったこと	
自分で試したこと	
AIへ入力した質問	
AIから得たヒント	
AIの説明を確認した方法	
次に同じ問題を解くときの注意点	

解答・解説

解答を見る前に

最低でも一度は、自分で値を追ってください。AIを使った場合も、最後はAIを閉じて解き直してから解答を確認します。

演習2-A

答え:10

実行した文	実行後の total
total ← 2	2
total ← total + 3	5
total ← total × 2	10

最後の行では、直前の値5を使って 5×2 を計算します。

演習2-B

```
work ← x
x ← y
y ← work
```

① x、② y、③ work

最初に x の値7を work へ退避してから、x を12へ更新します。

演習3-A

① 150、② 80、③ $\text{apple} \times 3 + \text{orange} \times 4$

合計金額は770円です。

演習3-B

```
taxIncluded ← price × (1 + rate)
表示する(taxIncluded)
```

税額だけを求める場合は $\text{price} \times \text{rate}$ ですが、税込価格には元の価格も含まれます。

演習4-A

```
x mod 2 = 0
```

2で割った余りが0なら偶数です。

演習4-B

① $\text{bmi} < 18.5$

② $\text{bmi} < 25$

最初の条件が成立しなかった時点で、 $\text{bmi} \geq 18.5$ であることが分かっています。そのため、二つ目では上限だけを確認できます。

bmi	結果
18.49	低体重
18.50	標準体重
24.99	標準体重
25.00	肥満

演習5-A

① 0、② 1、③ 10、④ $\text{total} + i$ 、⑤ $\text{total} \div 10$

合計は55、平均は5.5です。

演習5-B

処理①は5回、処理②は3回です。

処理②を実行するのは、 i が1、3、5の場合です。 i が6のとき、条件判定は行いますが、繰返し内部の処理は実行しません。

演習6-A

① 0、② 1、③ 10、④ $\text{total} + A[i]$

i は添字、 $A[i]$ はその位置に保存されている値です。

演習6-B

max は次のように変化します。



現在の max より大きい値が現れた場合だけ更新します。

演習7-A

最初の誤りは $i = 3$ の行です。 2×3 は6なので、実行後の total は6です。続く $i = 4$ では 6×4 を計算し、24になります。

演習8-A

① 0

② $A[i] \geq 5$

③ $\text{count} \leftarrow \text{count} + 1$

条件を満たした要素に出会うたびに、件数を1増やします。

演習8-B

① `A[i] > max`

② `A[i]`

③ `i`

> を使うため、同じ最大値が後から現れても更新せず、最初の位置が残ります。

演習8-C

```
for (iを1からnumまで1ずつ増やす)
  for (jを1からnumまで1ずつ増やす)
    "*"を横に表示する
  endfor
  改行する()
endfor
```

外側の繰返しが行数、内側の繰返しが一行に表示する * の個数を担当します。

研修後の学習ロードマップ

本書で扱った内容は、科目Bのアルゴリズム問題を読むための入口です。次の順番で学習を続けます。

段階	学習内容	到達目標
1	変数・代入・条件分岐・繰り返し	短いコードをトレースできる
2	一次元・二次元配列	添字と要素を区別できる
3	合計・件数・最大・最小	基本パターンを説明できる
4	線形探索・二分探索	探索範囲の変化を追える
5	基本的な整列	要素の交換を追える
6	スタック・キュー・リスト	データ構造の操作を理解できる
7	再帰・木・グラフ	呼出しと探索順を追える
8	科目B形式の総合問題	時間内に必要な値を追える

自習の一週間モデル

曜日	学習内容
1日目	概念説明を読み、例題をトレースする
2日目	基本問題をAIなしで解く
3日目	間違えた問題についてAIへ質問する
4日目	AIが作成した易しい類題を解く
5日目	本番形式の問題を時間を測って解く
6日目	間違いを「読解・条件・添字・計算・トレース」に分類する
7日目	AIを使わず、間違えた問題を解き直す

生成AIを使う場合

PDFファイルを添付できる生成AIに本書を読み込ませる場合は、質問に次の条件を付けます。

生成AIへコピーして使える指示文

登録された教材を優先して回答してください。

答えを直接示す前に、私がどこまで考えたかを確認してください。

最初はヒントを一つだけ提示してください。

擬似言語を説明するときは、変数の変化を表にしてください。

私の考えに誤りがある場合は、最初の誤りだけを指摘してください。

教材と異なる説明をする場合は、そのことを明示してください。

最後は必ずAIなしで解く

AIの説明を理解できても、本番でAIは使用できません。各単元の最後には、AIを閉じ、問題文と紙だけで同じ問題をもう一度解いてください。

学習記録

日付	学習範囲	自力で解けた問題	AIへ質問した内容	解き直し
				☐
				☐
				☐
				☐

参考資料

- 独立行政法人情報処理推進機構(IPA)「試験要綱・シラバスについて」
 - 独立行政法人情報処理推進機構(IPA)「基本情報技術者試験 科目Bのサンプル問題」
 - 独立行政法人情報処理推進機構(IPA)「SG・FE 公開問題」
-

本教材は内容確認用の初稿です。擬似言語の最終的な表記、演習量、難易度、解説の粒度は、研修対象者と実施時間に合わせて調整します。